CLASSICAL ML / DECISION REFERENCE

Which Model When

A working reference for classical ML model selection. Left side is for fast interview recall, right side for real project calls. No deep learning, no LLMs.

01 Start here: the 30-second decision

1. Is the target a **label or a number?** Label to classification, number to regression. No target to clustering or dimensionality reduction.
2. Need to **explain every prediction** to a stakeholder? Start with Linear/Logistic Regression or a single Decision Tree.
3. Want **best accuracy on tabular data** and can trade interpretability? Go straight to Gradient Boosting (XGBoost / LightGBM).
4. Dataset **small or noisy**, features many? Regularized linear models or SVM.
5. Only when tabular boosting underperforms do you reach for anything heavier.

02 Supervised: classification & regression

Linear / Logistic Regression

`baseline · interpretable · fast`

USE	Linear relationships, need coefficients you can explain, want a baseline before anything fancy.
SKIP	Strong non-linear patterns, heavy feature interactions.
WATCH	Scale features, check multicollinearity, regularize (L1/L2) when features outnumber clean signal.

Decision Tree

`interpretable · non-linear · unstable`

USE	Need a human-readable rule set, mixed feature types, non-linear splits, no scaling needed.
SKIP	You need stable, high accuracy. A single tree overfits easily.
WATCH	Prune or cap depth. Tiny data changes reshape the whole tree.

Random Forest

`robust · low-tuning · tabular`

USE	Strong default for tabular. Handles non-linearity and noise, minimal tuning, gives feature importance.
SKIP	Very large data where boosting wins, or when you need the single best score.
WATCH	Larger and slower than boosting at similar accuracy. Less interpretable than one tree.

Gradient Boosting (XGBoost / LightGBM)

`top tabular accuracy · tunable`

USE	The go-to for winning tabular performance, structured data, competitions, most production tabular tasks.
SKIP	Tiny datasets, or when full interpretability is mandatory.
WATCH	Sensitive to hyperparameters. Can overfit without early stopping and learning-rate control.

Support Vector Machine (SVM)

`small data · high-dim · margins`

USE	Small-to-medium datasets, high-dimensional spaces (text, genomics), clear margins. Kernels for non-linear.
SKIP	Large datasets (slow to train), or when you need probability outputs cheaply.
WATCH	Must scale features. Kernel + C + gamma tuning matters a lot.

k-Nearest Neighbors (kNN)

`no training · lazy · intuitive`

USE	Small datasets, low dimensions, quick baseline, recommendation-style similarity.
SKIP	Large or high-dimensional data. Prediction is slow and distances break down.
WATCH	Scale features, choose k carefully, suffers from the curse of dimensionality.

Naive Bayes

`text · fast · probabilistic`

USE	Text classification, spam filtering, high-dimensional sparse features, very fast baseline.
SKIP	Features are strongly correlated, or you need calibrated probabilities.
WATCH	The independence assumption is usually false but often works anyway.

Regularized Linear (Ridge / Lasso / ElasticNet)

`high-dim · feature selection`

USE	Many features, risk of overfitting. Lasso zeros out weak features; Ridge shrinks; ElasticNet blends.
SKIP	Clearly non-linear relationships that no feature engineering fixes.
WATCH	Tune the regularization strength via cross-validation, always scale first.

03 Unsupervised: clustering & structure

K-Means

`clustering · fast · spherical`

USE	You roughly know the number of clusters, groups are compact and similar in size.
SKIP	Irregular shapes, varying density, or unknown cluster count.
WATCH	Must set k, scale features, sensitive to initialization and outliers.

DBSCAN

`density · finds outliers · no k`

USE	Arbitrary cluster shapes, unknown cluster count, want automatic outlier detection.
SKIP	Clusters of very different densities, high dimensions.
WATCH	eps and min_samples are fiddly. Distance meaning degrades in high dimensions.

Hierarchical (Agglomerative)

`dendrogram · nested groups`

USE	Want a cluster hierarchy or dendrogram, small datasets, don't want to preset k.
SKIP	Large datasets, it scales poorly.
WATCH	Linkage choice (ward, average, complete) changes results significantly.

PCA

`dim-reduction · linear · preprocessing`

USE	Reduce dimensions, remove correlation, speed up models, visualize variance structure.
SKIP	You need interpretable features, or structure is non-linear (use t-SNE / UMAP to visualize).
WATCH	Scale first. Components are combinations, not original features.

04 Fast lookup by situation

SITUATION	FIRST REACH FOR	WHY
Tabular, want best accuracy	XGBoost / LightGBM	Consistently strongest on structured data
Need to explain to stakeholders	Logistic Reg / single Tree	Coefficients or readable rules
Quick strong baseline	Random Forest	Robust with almost no tuning
Text classification	Naive Bayes / Linear SVM	Fast on sparse high-dim features
Small, high-dimensional data	SVM	Works well when samples are few
Many features, overfitting	Lasso / ElasticNet	Shrinks and selects features
Group data, know cluster count	K-Means	Fast and simple
Group data, odd shapes / outliers	DBSCAN	No k needed, flags noise
Too many features to model	PCA	Compress before modeling
Tiny dataset, quick check	kNN	No training, intuitive

05 Interview one-liners

BIAS/VAR Linear = high bias, low variance. Deep tree / kNN(low k) = low bias, high variance. Ensembles cut variance.

BAGGING Random Forest: parallel trees on bootstrapped samples, reduces variance.

BOOSTING Sequential trees fixing prior errors, reduces bias. XGBoost/LightGBM.

SCALING Needed for SVM, kNN, K-Means, PCA, linear-with-regularization. Not for tree-based models.

IMBALANCE Class weights, resampling (SMOTE), and judge with precision/recall/F1 or PR-AUC, not accuracy.

L1 VS L2 L1 (Lasso) drives weights to zero (selection). L2 (Ridge) shrinks smoothly, keeps all.

`no free lunch` `baseline first` `CV everything` `leakage kills` `metric ≠ accuracy`

> **Most common real-world mistake:** data leakage. Fit scalers, encoders, and imputers on the training fold only, inside cross-validation, never on the full dataset before splitting.

CLASSICAL ML · MODEL SELECTION · v1 · for interview revision + project decisions